# Discription for our data base:

## Create tables:

In total we have 6 tables:

Customers, dishes, orders, customer_dish_table, customer_orders_table, order_dish_table.

**1. Customers Table**

**Description**: This table stores information about customers.

- **customer_id** (INTEGER): A unique identifier for each customer (primary key). Must be a positive integer.
- **customer_name** (VARCHAR(100)): The name of the customer. Cannot be null.
- **customer_phone** (VARCHAR(100)): The phone number of the customer. Cannot be null.
- **customer_address** (VARCHAR(100)): The address of the customer. Cannot be null.

---

**2. Orders Table**

**Description**: This table stores information about orders.

- **order_id** (INTEGER): A unique identifier for each order. Must be a positive integer.
- **order_date** (TIMESTAMP): The date and time when the order was placed. Cannot be null.

---

**3. Dishes Table**

**Description**: This table stores information about dishes.

- **dish_id** (INTEGER): A unique identifier for each dish. Must be a positive integer.
- **dish_name** (VARCHAR(100)): The name of the dish. Must be at least 3 characters long. Cannot be null.
- **dish_price** (DECIMAL): The price of the dish. Must be a positive decimal number. Cannot be null.
- **dish_is_active** (BOOLEAN): Indicates whether the dish is currently active. Cannot be null.

---

**4. Customers_Orders Table**

**Description**: This table stores the relationship between customers and their orders.such that each order can be associated with at most one customer.

- **customer_id** (INTEGER): The ID of the customer. References customers(customer_id). If a customer is deleted, this field is set to NULL.
- **order_id** (INTEGER): The ID of the order a primary key. References orders(order_id). If an order is deleted, the related records are also deleted.

---

**5. Order_Dish Table**

**Description**: This table stores the relationship between orders and dishes. such that each order can contain multiple dishes but each dish should be specified only once with the amount that was ordered per order, thus we put (dish_idm order_id) as primary key.

- **dish_id** (INTEGER): The ID of the dish. References dishes(dish_id). If a dish is deleted, the related records are also deleted.
- **order_id** (INTEGER): The ID of the order. References orders(order_id). If an order is deleted, the related records are also deleted.
- **amount** (INTEGER): The quantity of the dish ordered. Must be a positive integer.
- **price** (DECIMAL): The price of the dish at the time of the order. Must be a positive decimal number.

---

**6. Customer_Dishes Table**

**Description**: This table stores the relationship between customers and the dishes they like. such that each customer can order multiple dishes but each dish should be specified only once per customer, thus we put (dish_id, cust_id) as primary key.

- **customer_id** (INTEGER): The ID of the customer. References customers(customer_id). If a customer is deleted, the related records are also deleted.
- **dish_id** (INTEGER): The ID of the dish. References dishes(dish_id). If a dish is deleted, the related records are also deleted.
- **is_like** (BOOLEAN): Indicates whether the customer likes the dish. Cannot be null.

---

**Clear_tables():** This function is designed to delete all records from each table in the database. It contains SQL DELETE queries for the tables above.

**Drop_tables():** his function is responsible for dropping all the tables in the database, effectively deleting them. It contains SQL DROP TABLE queries for the tables above

---

# CRUD API functions:

1. **add_customer:**
   This query inserts a new customer into the customers table. If a customer with the same customer_id already exists, it does nothing.

```
INSERT INTO customers (customer_id, customer_name, customer_phone,
customer_address)
VALUES ('{customer_id}',
'{customer_name}','{customer_phone}','{customer_address}')
ON CONFLICT (customer_id) DO NOTHING
```

2. **get_customer:**
   This query retrieves a customer's details from the customers table based on the customer_id.

```
    f"""SELECT * FROM customers WHERE customer_id = '{customer_id}'"""
```

3. **delete_customer:**
This query deletes a customer from the customers table based on the customer_id. It returns the customer_id if the customer existed, otherwise returns NULL.

```
f"""
    DELETE FROM customers
    WHERE customer_id = {customer_id}
    RETURNING CASE WHEN EXISTS (SELECT 1 FROM customers WHERE customer_id =
{customer_id})
                  THEN {customer_id}
                  ELSE NULL
              END AS customer_id"""
```

---------- ORDERS----------

4. **add_order:**
This query inserts a new order into the orders table. If an order with the same order_id already exists, it does nothing.

```
# Add the order to the database with ON CONFLICT clause
add_query = f"""
    INSERT INTO orders (order_id, order_date)
    VALUES ('{order_id}', '{order_timespam}')
    ON CONFLICT (order_id) DO NOTHING
"""
```

5. **get_order:**
This query retrieves an order's details from the orders table based on the order_id.

```
query = f"""SELECT * FROM orders WHERE order_id =
'{order_id}'"""
```

6. **delete_order:**
This query deletes an order from the orders table based on the order_id. It returns the order_id if the order existed, otherwise returns NULL.

```
delete_query = f"""
    DELETE FROM orders
    WHERE order_id = {order_id}
    RETURNING CASE WHEN EXISTS (SELECT 1 FROM orders WHERE
order_id = {order_id})
                  THEN {order_id}
                  ELSE NULL
              END AS customer_id"""
```

---------- DISHES ----------

7. **add_dish:**
This query inserts a new dish into the dishes table. If a dish with the same dish_id already exists, it does nothing.

```
add_query = f"""
    INSERT INTO dishes (dish_id, dish_name,
dish_price,dish_is_active)
    VALUES ('{dish_id}', '{dish_name}', '{dish_price}',
'{dish_is_active}')
```

```
    ON CONFLICT (dish_id) DO NOTHING
"""
```

8. **get_dish**:
   This query retrieves a dish's details from the dishes table based on the dish_id

```
query = f"""SELECT * FROM dishes WHERE dish_id =
'{dish_id}'"""
```

9. **update_dish_price:**
   This query updates the price of a dish in the dishes table based on the dish_id.

```
update_query = f"""UPDATE dishes SET dish_price = '{price}'
WHERE dish_id = '{dish_id}'"""
```

10. **update_dish_active_status:**
    This query updates the active status of a dish in the dishes table based on the dish_id.

```
update_query = f"""UPDATE dishes SET dish_is_active =
'{is_active}' WHERE dish_id = '{dish_id}'"""
```

---------- CUSTOMER – ORDER – RELATION ----------

11. **customer_placed_order:**
    This query inserts a record into the customers_orders_table indicating that a customer placed an order. If the order already exists in the table, it does nothing.

```
customer_placed_order_add = f"""
    INSERT INTO customers_orders_table (customer_id,
order_id)
    SELECT
        {customer_id},
        {order_id}
    WHERE NOT EXISTS (
        SELECT 1
        FROM customers_orders_table
        WHERE order_id = {order_id}
    )
"""
```

12. **get_customer_that_placed_order:**
    This query retrieves the customer_id of the customer who placed a specific order from the customers_orders_table.

```
# Query to get the customer_id associated with the given
order_id
query = f"""
    SELECT customer_id
    FROM customers_orders_table
    WHERE order_id = {order_id}
"""
```

13. **order_contains_dish:**

This query inserts a record into the order_dish_table indicating that an order contains a specific dish. If the combination of dish and order already exists in the table, it does nothing.

```
add_query = f"""
    INSERT INTO order_dish_table (dish_id,
order_id,amount,price)
    SELECT
        {dish_id},
        {order_id},
        {amount},
        {price}
    WHERE NOT EXISTS (
        SELECT 1
        FROM order_dish_table
        WHERE dish_id = {dish_id} AND order_id = {order_id}
    )
"""
```

14. **order_does_not_contain_dish:**
This query deletes a record from the order_dish_table indicating that an order no longer contains a specific dish.

```
delete_query = f"""
    DELETE FROM order_dish_table
    WHERE order_id = {order_id} AND dish_id = {dish_id}
"""
```

15. **get_all_order_items:**
This query retrieves all dishes(in ASC order) associated with a specific order from the order_dish_table.

```
query = f"""
    SELECT order_dish_table.dish_id,
order_dish_table.amount,order_dish_table.price
    FROM order_dish_table
    JOIN dishes ON order_dish_table.dish_id = dishes.dish_id
    WHERE order_dish_table.order_id = {order_id}
    ORDER BY order_dish_table.dish_id ASC
"""
```

16. **customer_likes_dish:**
This query inserts a record into the customer_dish_table indicating that a customer likes a specific dish.

```
add_query = f"""
    INSERT INTO customer_dish_table (customer_id, dish_id,
is_like)
    VALUES ('{cust_id}', '{dish_id}','{is_like}')
"""
```

17. **customer_dislike_dish:**
This query updates a record in the customer_dish_table indicating that a customer dislikes a specific dish.

```
update_query = f"""UPDATE customer_dish_Table
 SET is_like = '{is_like}'
 WHERE customer_dish_table.dish_id = '{dish_id}'
 AND customer_dish_table.customer_id = '{cust_id}' """
```

18. **get_all_customer_likes:**
    This query retrieves all dishes liked by a specific customer from the customer_dish_table and dishes table.

```
query = f"""
    SELECT dishes.dish_id, dishes.dish_name,
dishes.dish_price, dishes.dish_is_active
    FROM dishes
    JOIN customer_dish_table ON customer_dish_table.dish_id
     = dishes.dish_id
    WHERE customer_dish_table.customer_id = '{cust_id}' AND
customer_dish_table.is_like = TRUE
    ORDER BY customer_dish_table.dish_id ASC
"""
```

---

# BASIC FUNCTIONS AND THEIR VIEWS:

1. **def get_order_total_price(order_id: int) -> float:**

This query retrieves the total price for a specific order from the order_total_prices view

```
query = f"""SELECT total_price FROM order_total_prices WHERE
order_id = '{order_id}'"""
```

views for the function:

- ordal_total_prices view:
  calculate the total price for each order based on the prices and amounts of dishes included in that order.

---

**2. def get_max_amount_of_money_cust_spent(cust_id: int) -> float:**

This query finds the maximum total price of any order placed by the customer with the given cust_id.

It joins the customers_orders_table (which maps customers to their orders) with the order_total_prices view used above to get the total prices of those orders.

The COALESCE(MAX(otp.total_price), 0) ensures that if there are no orders, the result will be 0.

```
max_amount_query = f"""
    SELECT COALESCE(MAX(otp.total_price), 0) AS max_spent
    FROM customers_orders_table cot
    JOIN order_total_prices otp ON cot.order_id = otp.order_id
    WHERE cot.customer_id = {cust_id};
"""
```

---

3. **def get_most_expensive_anonymous_order() -> Order:**

This query selects the order_id of the most expensive order from the anonymoes table.

It orders the results by total_price in descending order and by order_id in ascending order.

The LIMIT 1 clause ensures that only the single most expensive order is selected.

```
max_amount_query = f"""
    SELECT anonymoes.order_id
    FROM anonymoes
    ORDER BY anonymoes.total_price DESC, anonymoes.order_id ASC
    LIMIT 1;
  """
```

views for the function:

- anonymoes_view:

  The anonymoes_view creates a view named anonymoes that includes only the orders with no customer ID from the total_orders table.
  (when deleting a customer from the customer table we set the cust_id in this table to be NUL)

  This view helps to easily query anonymous orders by selecting order_id and total_price where customer_id is NULL.

---

4. **def is_most_liked_dish_equal_to_most_purchased() -> bool:**

This query checks if the most liked dish is the same as the most purchased dish. It does this by querying the dish_comparison table view and returns True if there's a match and False otherwise.

```
query = f"""
SELECT
    COUNT(*) > 0 AS are_equal
FROM
    dish_comparison;
    """
```

views for the function:

- View for Most Liked Dishes:

  This view calculates the total number of likes for each dish.
  This view sums up the total likes (`total_likes`) for each dish (`dish_id`) from the `customer_dish_table`.

- View for Most Purchased Dishes:

  This view calculates the total number of purchases for each dish.

  This view sums up the total purchases (`total_purchase`) for each dish (`dish_id`) from the `order_dish_table`.

- View for Comparing Most Liked and Most Purchased Dishes:

  This view checks if the most liked dish is also the most purchased dish.

  This view joins the top liked dish (`most_liked_dish`) and the top purchased dish (`most_purchased_dish`) to see if they are the same dish by comparing their `dish_id`.

---

# ADVANCED API FUNCTIONS AND THEIR VIEWS:

**1. def get_customers_ordered_top_5_dishes() -> List[int]:**

This query to retrieve customer IDs from the customers_ordered_top_5_dishes view, ordering them by customer_id in ascending order.

```
query = f"""
SELECT customer_id
FROM customers_ordered_top_5_dishes
ORDER BY customer_id ASC;
"""
```

views for the function:

- most_liked_dish_1_view

  This view calculates the total likes for each dish.
  It Summarizes the total likes for each dish.

- top_5_most_liked_dishes_view

  This view selects the top 5 most liked dishes based on the total likes.

  It identifies the top 5 dishes with the highest number of likes.

  - Orders dishes by total_likes in descending order.
  - Limits the results to the top 5 dishes.

- customers_ordered_top_5_dishes_view

  This view selects customers who have ordered all of the top 5 most liked dishes.

  Finds customers who have ordered all top 5 most liked dishes.

  - Joins customers_orders_table and order_dish_table on order_id.
  - Filters the results to include only orders for dishes in the top_5_most_liked_dishes.
  - Groups by customer_id and dish_id.
  - Selects only customers who have ordered all 5 distinct dishes (checked by HAVING COUNT(DISTINCT dish_id) = 5).

---

2. **def get_non_worth_price_increase() -> List[int]:**

This query retrieves a list of distinct dish IDs from the comparison view where it might be deemed non-beneficial to increase the price.

```
query = f"""
SELECT DISTINCT dish_id
FROM comparison
ORDER BY dish_id;
"""
```

views for the function:

- active_dishes_view:
  Filters active dishes and their current prices.

- order_dish_amounts_view:
  Calculates average ordered amounts for each dish and price combination.

  It computes the average amount ordered for each dish_id and price combination from the order_dish_table

- current_profits_view:
  Computes current average profits for active dishes.
  It joins active_dishes with order_dish_amounts to calculate the average profit (avg_amount * current_price) for each active dish

- previous_profits_view:
  Computes previous average profits for active dishes at lower prices.
  It joins active_dishes with order_dish_amounts to calculate the average profit (avg_amount * price) for each active dish at previous prices.

- comparison_view:
  Identifies dishes where the current profit is less than the previous profit, indicating non-beneficial price increases.

---

3. def get_total_profit_per_month(year: int) -> List[Tuple[int, float]]:

The query selects the month and total profit for each month in the specified year.

It performs a LEFT JOIN between a months table (which contains all months) and an order_monthly_profits table (which contains the monthly profits for the specified year).

If there is no profit data for a month, it returns 0 as the total profit for that month.

```
query = f"""
SELECT
    m.month,
    COALESCE(omp.total_profit, 0) AS total_profit
FROM
    months m
LEFT JOIN
    order_monthly_profits omp
ON
    m.month = omp.month AND omp.year = {year}
ORDER BY
    m.month DESC;
"""
```

views for the function:

- order_monthly_profits_view:

  This view calculates the total profit for each month and year.

  It extracts the month and year from the order_date column in the orders table. and then it sums the total profit, which is calculated as price * amount, from the order_dish_table joined with the orders table.The results are grouped by month and year.

- months_view:
  This view generates a series of numbers from 1 to 12, representing the months of the year.It uses the generate_series function to create a list of months from 1 to 12.

---

4. def get_potential_dish_recommendations(cust_id: int) -> List[int]:

Selects dish IDs from the potential_recommendations view where the target_customer_id matches the given customer ID.

Orders the results by dish_id in ascending order.

It returns a list of dish IDs that are potential recommendations for the specified customer.

```
query = f"""
    SELECT
        pr.dish_id
    FROM
        potential_recommendations pr
    WHERE
        pr.target_customer_id = {cust_id}
    ORDER BY
        pr.dish_id ASC;
    """
```

views for the function:

- **customer_liked_dishes View**

  This view lists all dishes that customers have liked.

  It looks at the table where customer interactions with dishes are recorded (customer_dish_table) and selects only the interactions where the customer indicated they liked the dish.

  This view finds customers with similar tastes based on liked dishes. It identifies pairs of customers who have liked at least three of the same dishes.

  It joins the customer_liked_dishes view with itself to compare different customers. If two customers have liked the same dishes, they are considered similar. The view ensures that it only pairs different customers (i.e., a customer is not paired with themselves) and only includes pairs who have liked at least three of the same dishes.

- **potential_recommendations**

This view generates dish recommendations for customers based on the preferences of similar customers.

Recommends new dishes to customers based on what their similar customers have liked but they haven't yet tried.

It uses the similar_customers view to find customers with similar tastes. Then, it looks at the dishes these similar customers have liked. For each target customer, it excludes any dishes they have already liked and recommends the remaining dishes.

---